



Today's agenda

↳ insertion sort

↳ quick sort



AlgoPrep



Q) Given $arr[N]$, first $N-1$ elements are sorted, sort entire $arr[]$.

Ex: $arr[] = \{2, 6, 10, 14, 20, 4\}$
 $\quad \quad \quad 2, 4, 6, 10, 14, 20$

//idea 1

↳ $Arrays.sort(arr)$

T.C: $O(N \log N)$

//idea 2

$arr[] = \{2, 6, 10, 14, 20, 4\}$
 $\quad \quad \quad 4, 6, 10, 14, 20$

//pseudo code

T.C: $O(N)$

S.C: $O(1)$

```
void insertElement (int arr[N]) {
    for (int j = N-2; j >= 0; j--) {
        if (arr[j] > arr[j+1]) {
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
        else {
            break;
        }
    }
}
```



Q) Insertion Sort

↳ Given $arr[N]$, sort it using insertion step.

$arr[6] : \{ \overset{0}{10} \overset{1}{3} \overset{2}{6} \overset{3}{8} \overset{4}{2} \overset{5}{5} \}$

Step 1 : $\{ \overset{0}{10} \overset{1}{3} \overset{2}{6} \overset{3}{8} \overset{4}{2} \overset{5}{5} \}$
 $\{ \overset{0}{3} \overset{1}{10} \overset{2}{6} \overset{3}{8} \overset{4}{2} \overset{5}{5} \}$

Step 2 : $\{ \overset{0}{3} \overset{1}{10} \overset{2}{6} \overset{3}{8} \overset{4}{2} \overset{5}{5} \}$
 $\{ \overset{0}{3} \overset{1}{6} \overset{2}{10} \overset{3}{8} \overset{4}{2} \overset{5}{5} \}$

Step 3 : $\{ \overset{0}{3} \overset{1}{6} \overset{2}{10} \overset{3}{8} \overset{4}{2} \overset{5}{5} \}$
 $\{ \overset{0}{3} \overset{1}{6} \overset{2}{8} \overset{3}{10} \overset{4}{2} \overset{5}{5} \}$

Step 4 : $\{ \overset{0}{3} \overset{1}{6} \overset{2}{8} \overset{3}{10} \overset{4}{2} \overset{5}{5} \}$
 $\{ \overset{0}{2} \overset{1}{3} \overset{2}{6} \overset{3}{8} \overset{4}{10} \overset{5}{5} \}$

Step 5 : $\{ \overset{0}{2} \overset{1}{3} \overset{2}{6} \overset{3}{8} \overset{4}{10} \overset{5}{5} \}$
 $\{ \overset{0}{2} \overset{1}{3} \overset{2}{5} \overset{3}{6} \overset{4}{8} \overset{5}{10} \}$

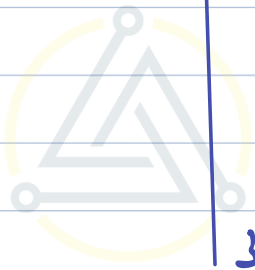


11Pseudo code

T.C: $O(N^2)$

S.C: $O(1)$

```
void insertionSort (int arr[N]) {  
    for (int i = 1; i < N; i++) {  
        for (int j = i - 1; j >= 0; j--) {  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
            else {  
                break;  
            }  
        }  
    }  
}
```



AlgoPrep



Tooling

$i=1$

arr[6] : { 10 3 6 8 2 5 }
 { 3 10 6 8 2 5 }

```
void insertionSort (int arr[6]) {
```

```
    for (int i=1; i<N; i++) {
```

```
        for (int j=i-1; j>=0; j--) {
```

```
            if (arr[j] > arr[j+1]) {
```

```
                int temp = arr[j+1];  
                arr[j+1] = arr[j];  
                arr[j] = temp;
```

```
            } else {  
                break;
```

```
        }
```

```
    }
```

$i=2$

{ 3 10 6 8 2 5 }
 6 10
{ 3 6 10 8 2 5 }

$i=3$

{ 3 6 10 8 2 5 }
 8 10
{ 3 6 8 10 2 5 }

$i=4$

{ 3 6 8 10 2 5 }
 2 6 8 10
 3 6 8 10
{ 2 3 6 8 10 5 }



- Q) Given $arr[n]$ elements, rearrange the array so that
- $arr[0]$ should go to correct sorted position.
 - all elements $\leq arr[0]$ leftside of $arr[0]$.
 - all elements $> arr[0]$ rightside of $arr[0]$.

$arr[11] = \{ 10, 3, 8, 15, 6, 12, 2, 18, 7, 15, 14 \}$

↓

18 15 14 15 12

10

6 3 2 8 7

//idea1

↳ Sort the array

T.C: $O(n \log n)$

//idea2

$arr[11] = \{ 10, 3, 8, 15, 6, 12, 2, 18, 7, 15, 14 \}$

↓

18 15 14 15 12

10

6 3 2 8 7

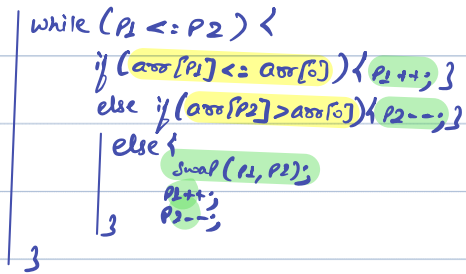
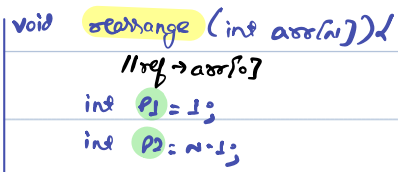


// Pseudo Code

```
void rearrange (int arr[N]) {  
    // ref → arr[0]  
    int p1 = 1;  
    int p2 = N-1;  
  
    while (p1 <= p2) {  
        if (arr[p1] <= arr[0]) { p1++; }  
        else if (arr[p2] > arr[0]) { p2--; }  
        else {  
            swap (p1, p2);  
            p1++;  
            p2--;  
        }  
    }  
  
    swap (0, p2);  
}
```

T.C: $O(N)$

S.C: $O(1)$



```
swap(0, p2);
```

13





Q) Given $arr[n]$ elements and (s, e) , rearrange the subarray so that

→ $arr[s]$ should go to correct position in (s, e)

→ all elements $\leq arr[s]$ leftside of $arr[s]$.

→ all elements $> arr[s]$ rightside of $arr[s]$.

$\{2, 7\}$
 $arr[11] = \{ 10, 3, 8, 15, 6, 12, 2, 18, 7, 15, 14 \}$

Diagram showing the partitioning process: $P1$ points to index 3 (value 8) and $P2$ points to index 7 (value 18). Elements less than or equal to 8 are on the left, and elements greater than 8 are on the right.

```
int rearrange (int arr[n], int s, int e) {
```

```
    // ref → arr[s]
```

```
    int P1 = s + 1;
```

```
    int P2 = e;
```

```
    while (P1 <= P2) {
```

```
        if (arr[P1] <= arr[s]) { P1++; }
```

```
        else if (arr[P2] > arr[s]) { P2--; }
```

```
        else {
```

```
            swap(P1, P2);
```

```
            P1++;
```

```
            P2--;
```

```
        }
```

```
    }
```

```
    swap(s, P2);
```

```
    return P2;
```

```
}
```



Quick Sort

↳ Given an `arr[]`, sort the array.

[x _ _ _ _]

[_ _ _ x _ _]
 $< x$ $> x$

T.C: $O(N^2)$

S.C: $O(N)$

```
void QuickSort (int arr[], int s, int e) {  
    if (s >= e) { return; }
```

```
    int p = rearrange(arr, s, e);
```

```
    QuickSort (arr, s, p-1);
```

```
    QuickSort (arr, p+1, e);
```

}



```
void quicksort (int arr[], int s, int e) {  
    1 if (s >= e) { return; }  
    2 int p = rearrange (arr, s, e);  
    3 quicksort (arr, s, p-1);  
    4 quicksort (arr, p+1, e);  
}
```

arr[10]: { 18 14 6 3 11 8 23 20 31 27 }
{ 3 6 8 11 14 18 23 20 31 27 }

{ 14 6 3 11 8 18 23 20 31 27 }
{ 3 6 8 11 14 18 23 20 31 27 }

{ 3 6 8 11 14 18 23 20 31 27 }

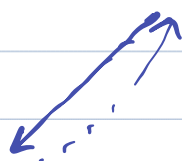
{ 6 8 3 11 14 18 23 20 31 27 }
{ 3 6 8 11 14 18 23 20 31 27 }

{ 3 6 11 8 14 18 23 20 31 27 } { 3 6 11 8 14 18 23 20 31 27 }
{ 3 6 8 11 14 18 23 20 31 27 }

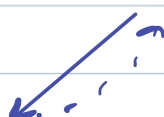
{ 3 6 8 11 14 18 23 20 31 27 } { 3 6 8 11 14 18 23 20 31 27 }



arr[5]: $\{ \overset{s \rightarrow p}{\underset{\uparrow}{2}} \ 4 \ 5 \ 6 \ 7 \} \rightarrow O(N)$



$\{ \overset{s \rightarrow p}{\underset{\uparrow}{2}} \ \overset{s \rightarrow p}{\underset{\uparrow}{4}} \ 5 \ 6 \ 7 \} \rightarrow O(N)$



$\{ \overset{s \rightarrow p}{\underset{\uparrow}{2}} \ \overset{s \rightarrow p}{\underset{\uparrow}{4}} \ \overset{s \rightarrow p}{\underset{\uparrow}{5}} \ 6 \ 7 \} \rightarrow O(N)$

double Pivoted
Quick Sort

Arrays.sort (int[] arr); $\rightarrow O(N^2)$

Arrays.sort (Integer[] arr); $\rightarrow O(N \log N)$
{merge sort}

arr[] $\rightarrow \left(\overset{2^p}{\underset{\uparrow}{0}} \ 1 \right) \left(2 \ 4 \ 5 \right)$



↳ randomize the reference pointer in Quick Sort.

```
void QuickSort (int arr[], int s, int e) {  
    if (s >= e) { return; }
```

```
    int p = rearrange (arr, s, e);
```

```
    QuickSort (arr, s, p-1);
```

```
    QuickSort (arr, p+1, e);
```

```
Random m = new Random();
```

T.C: $O(N \log N)$

S.C: $O(\log N)$

```
int rearrange (int arr[], int s, int e) {  
    int r = m.nextInt(s, e);  
    swap(r, s);  
    // arr[s] is reference  
    int p1 = s+1;  
    int p2 = e;  
    while (p1 <= p2) {  
        if (arr[p1] <= arr[s]) { p1++; }  
        else if (arr[p2] > arr[s]) { p2--; }  
        else {  
            swap(p1, p2);  
            p1++;  
            p2--;  
        }  
    }  
    swap(s, p2);  
    return p2;  
}
```